AutomaGuard: Compiled Temporal Policy Enforcement for Production AI Agents

Scott Davis
California State University, Sacramento
Sacramento, USA
scott@ScottsSecondAct.com

Abstract

AI agents that invoke external tools, access sensitive data, and operate with minimal human supervision require safety guarantees stronger than best-effort guardrails. We present **AutomaGuard**, a formal policy enforcement engine that provides mathematical guarantees—not probabilistic estimates—that agent behavior complies with specified constraints. AutomaGuard comprises three components: (1) the **Aegis Policy Language**, a domain-specific language in which policies are expressed as typed rules and temporal invariants; (2) an **ahead-of-time compiler** that translates Aegis source to deterministic state machines encoded in a compact bytecode format; and (3) a **Rust runtime verifier** that intercepts agent events, evaluates compiled rules, advances automaton state, and enforces sliding-window rate limits—all within a sub-10 ms per-event latency budget.

Unlike static workflow analyzers, AutomaGuard evaluates the *actual* execution trace of a running agent, catching violations that only manifest from runtime data (e.g., PII in an LLM response, a specific tool-call sequence not predictable from the graph topology). Unlike runtime guardrails that interpret policy at request time, AutomaGuard compiles policies to state machines ahead of deployment, eliminating parsing overhead and enabling a hard latency guarantee. Benchmarks on a realistic policy (10 rules, 2 state machines, 1 rate limiter) show a mean evaluation latency of 2.4 µs on the full evaluation path—a factor of 4,166× under the 10 ms per-event budget. A Python SDK with LangChain and OpenAI integrations enables drop-in enforcement with three lines of user code.

Keywords

AI agents, policy enforcement, formal verification, temporal logic, runtime monitoring, domain-specific language, Rust, safety

ACM Reference Format:
Scott Davis. 2026. AutomaGuard: Compiled Temporal Policy Enforcement for Production AI Agents. In . ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The deployment of AI agents in production settings introduces a qualitatively different risk profile from traditional software. Where a conventional application executes a deterministic program, an agent driven by a large language model (LLM) operates by selecting and invoking tools dynamically in response to inputs whose distribution is open-ended. A single agent session may traverse tool-call sequences that were never tested, involving sensitive resources such as customer PII, financial accounts, or privileged infrastructure APIs.

Existing approaches to constraining agent behavior fall into two categories, each with fundamental limitations:

Runtime guardrails (interpreted). Tools such as NeMo Guardrails [9], LlamaGuard [5], and Guardrails AI intercept individual outputs and evaluate them against rules expressed as prompts, classifiers, or regular expressions. These tools operate on a per-call basis: they cannot reason about *sequences* of events or enforce invariants that span multiple turns. Moreover, they re-parse or re-run inference at every evaluation, introducing latency that scales with policy complexity and model size.

Static workflow analyzers. Recent work [13] extracts graph models from agent frameworks and applies pre-deployment checks against temporal safety policies compiled to deterministic finite automata. Static analysis provides path-exhaustive guarantees but is fundamentally limited to properties derivable from the graph topology. It cannot catch violations conditioned on runtime values—for example, that the *content* of a specific API response triggers a forbidden downstream action—and offers no enforcement mechanism during live execution.

AutomaGuard occupies a third position: **compiled runtime enforcement**. Policies are expressed in a typed DSL with first-class temporal operators, compiled ahead of deployment to state machines, and evaluated against the live event stream at sub-microsecond latency per event. This design provides:

(1) **Sequence-level guarantees.** Temporal invariants expressed with `always`, `eventually`, `never`, and `until` compile to automata that detect violations spanning arbitrary-length event sequences—the moat that per-call guardrails cannot cross.
(2) **Hard latency bounds.** Because there is no policy parsing at evaluation time, the latency is determined entirely by rule evaluation and state-machine stepping—operations that are $O(\text{rules} + \text{automata})$ in the compiled policy, not $O(\text{policy source size})$.

(3) **Runtime data sensitivity.** Policies can condition on event field values, PII markers, tool argument contents, and aggregated statistics, all of which are only known at runtime.
(4) **Append-only audit trail.** Every verdict, with its triggering event, matched rules, automaton transitions, and evaluation latency, is logged immutably.

This paper makes the following contributions:

- The design and implementation of the **Aegis Policy Language**: syntax, type system with subtyping, temporal operators, rate-limit declarations, and policy inheritance (§3).
- A **compilation pipeline** from Aegis source to verified IR and compact `.aegisc` bytecode, including the state-machine construction algorithm for temporal invariants (§4).
- A **Rust runtime verifier** with allocation-minimal evaluation, sliding-window rate limiters, and an append-only ring-buffer audit log (§5).
- A **Python SDK** providing drop-in enforcement for LangChain and OpenAI agents via pyo3 bindings (§6).
- **Evaluation** demonstrating 2.4 µs mean latency on a realistic policy—a factor of 4,166× under the 10 ms budget (§7).

2 Motivation and Threat Model

2.1 Representative Agent Threats

We motivate the design with two attack patterns that appear in the `research_guard.aegis` and `data_exfiltration_guard.aegis` example policies included in the AutomaGuard distribution.

Indirect prompt injection via action sequences. An adversarial document instructs an agent to (a) search for sensitive files, then (b) email their contents to an external address. Each action in isolation may be permitted by per-call rules—searching and emailing are both legitimate capabilities. Only the *sequence* "file access of sensitive material followed by email to external recipient" is the violation. A per-call guardrail cannot detect this pattern; a temporal invariant over the event stream can.

PII exfiltration via tool arguments. An agent processing customer records may inadvertently include PII in the arguments to a third-party tool call. The violation is conditioned on the *content* of a specific runtime field, not on the workflow topology. Static analysis cannot detect it; a runtime rule that inspects tool-call arguments can.

2.2 Threat Model

AutomaGuard targets the following threats:

T1 Developer mistakes. Agent developers write policies that compile successfully but contain logical gaps (e.g., missing human-gate on a sensitive path). The compiler's type checker and diagnostic system surface these at compile time.

T2 Adversarial sequences. An adversary (via prompt injection or tool-call manipulation) orchestrates a sequence of actions that individually pass per-call checks but collectively violate a policy. Temporal invariants catch these.

T3 Rate-based abuse. An agent is induced to call an external API or read sensitive data at a rate that reveals information or causes cost overruns. Sliding-window rate limiters enforce budgets.

T4 Runtime data violations. A policy condition depends on the *value* of a field known only at runtime (PII marker, classification level, response content). Runtime expression evaluation catches these.

AutomaGuard does not address semantic correctness of LLM outputs (factuality, intent), non-deterministic agent topology, or threats arising from the model weights themselves.

3 The Aegis Policy Language

Aegis is a statically typed, declarative DSL for expressing agent behavioral constraints. A policy is a named collection of rules, constraints, proofs, and (optionally) type and function definitions. Policies may extend a base policy, inheriting its rules and adding or overriding members.

3.1 Syntax Overview

Listing 1 shows a minimal policy demonstrating the core language elements.

```
policy DataGuard {
  // Rate limit: max 100 tool calls per minute
  rate_limit tool_calls: 100 per 1m

  // Per-event rule
  on tool_call {
    when event.tool_name == "drop_table"
    deny "DDL␣operations␣are␣prohibited"
    severity critical
  }

  on external_request {
    when !is_approved(event.endpoint)
    audit "Unapproved␣endpoint"
    severity high
  }

  // Temporal invariant: PII never reaches a tool
      call
  proof PIIContainment {
    invariant NoPIIInArgs {
      never(any(event.arguments, arg =>
        arg.classification == "PII"))
    }
  }
}
```

Listing 1: A minimal Aegis policy.

3.2 Event Model

Every agent action is represented as an *event*: a typed pair (*event_type*, *fields*) where *fields* is a string-keyed map of runtime-typed values. The built-in event types and their field schemas are:

Event type	Key fields
`tool_call`	`tool_name`, `arguments`, `result`
`external_request`	`endpoint`, `method`, `body`
`data_access`	`resource`, `classification`
`message`	`role`, `content`
`file_access`	`path`, `operation`
`code_execution`	`language`, `code`

3.3 Type System

Aegis has a structural type system with subtyping. The primitive types are `int`, `float`, `bool`, `string`, and `duration` (e.g., `5ms`, `10s`, `24h`). Collection types include `List<T>`, `Map<K,V>`, and `Set<T>`. Union types (`A | B`) are supported. User-defined struct types are declared with `type`.

Subtyping rules are:

- `never` is the bottom type (subtype of all types).
- `int` $\sqsubseteq$ `float` (numeric widening).
- $T \sqsubseteq T \mid U$ and $U \sqsubseteq T \mid U$ (union introduction).
- Collections are covariant: `List<int>` $\sqsubseteq$ `List<float>`.

3.4 Rule Clauses

A rule is triggered when an event matches its `on` declaration. The body consists of:

- Zero or one `when` clause (a boolean expression; omitting it means the rule fires on every matching event).
- One or more verdict clauses: `allow`, `deny`, `audit` (log but allow), or `redact` (allow but sanitize output).
- Optional action clauses: `log`, `notify`, `escalate`, `block`, `tag`.
- An optional severity annotation: `critical`, `high`, `medium`, `low`, `info`.

When multiple rules fire on a single event, `deny` takes precedence over all other verdicts; among equal-precedence verdicts the first-fired rule wins.

3.5 Temporal Operators

Temporal invariants are placed inside `proof` blocks. The language supports the following operators, corresponding to the stated LTL formulae over the event stream $\sigma = e_0, e_1, e_2, \ldots$:

Aegis syntax	LTL formula	Intuition
`always(`ϕ`)`	$\Box\phi$	ϕ holds on every event
`never(`ϕ`)`	$\Box\neg\phi$	ϕ never holds
`eventually(`ϕ`) within T`	$\Diamond_{\leq T}\phi$	ϕ holds within deadline T
`until(`ϕ`, `ψ`)`	$\phi\,\mathcal{U}\,\psi$	ϕ holds until ψ
`before(`ϕ`, `ψ`)`	$\neg\psi\,\mathcal{U}\,\phi$	ϕ must precede ψ
`after(`ϕ`, `ψ`)`	$\Box(\psi \Rightarrow \mathcal{X}\phi)$	after ψ, ϕ must hold next
`next(`ϕ`)`	$\mathcal{X}\phi$	ϕ holds on the next event

The expressions ϕ and ψ are full Aegis boolean expressions over the current event's fields, supporting quantifiers (`all`, `any`, `none`, `exists`), string predicates (`contains`, `matches`, `starts_with`), arithmetic, and user-defined functions.

Composing temporal and data predicates. Aegis's key expressive advantage over approaches that restrict to fixed property templates is the ability to compose temporal operators with arbitrary data predicates. For example, the following invariant detects the indirect-injection attack from §2:

```
proof ExfiltrationDetection {
  invariant NoPIIFollowedByExternalSend {
    never(
      after(
        event.event_type == "external_request"
          && event.endpoint matches r".*@.*",
        event.event_type == "data_access"
          && event.classification == "PII"
      )
    )
  }
}
```

Listing 2: Sequence-level exfiltration detection.

3.6 Rate Limits and Quotas

Rate limits are declared at policy scope:

```
rate_limit tool_calls:         100 per 1m
rate_limit external_requests:  20  per 5m
quota      data_reads:         10000 per 1h
```

The runtime enforces these with sliding-window counters (§5.4), which prevents burst-at-window-boundary exploits that fixed-window counters are susceptible to.

3.7 Policy Inheritance

A policy may extend exactly one base policy using the `extends` keyword. Inheritance is single (no diamonds). The derived policy inherits all rules, constraints, and proof blocks from the base and may add new members or override specific rules by name. This enables a common pattern of a strict production policy with a more permissive development variant:

```
policy DevMode extends DataGuard {
  on tool_call {
    when event.environment == "development"
    allow
  }
}
```

4 Compilation Pipeline

Figure 1 shows the five-stage compilation pipeline from Aegis source to `.aegisc` bytecode.

4.1 Parsing

The parser is generated from a PEG grammar using the `pest` crate. The grammar is organized into three layers:

(1) **Lexical**: keywords, identifiers, literals (bool, int, float, duration, string, raw string `r"..."`, regex `r/.../`).
(2) **Expressions**: full expression grammar with precedence climbing (binary operators, unary, predicates, quantifiers, lambdas, `match` expressions, temporal sub-expressions).
(3) **Declarations**: `policy`, `proof`, `type`, `let`, `def`, `import`, with annotation support (`@name(args)`).

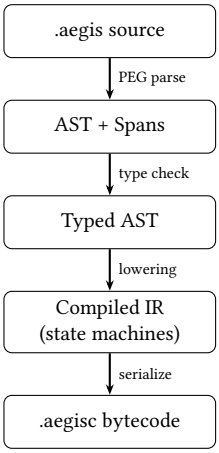

Figure 1: Aegis compilation pipeline.

Every AST node carries a `Spanned<T>` wrapper recording its byte range in the source file. All diagnostic messages produced by later passes include source spans, enabling IDE-quality error reporting.

4.2 Type Checking

The type checker runs in two passes to support forward references:

(1) **Registration pass**: Collect all top-level type and function declarations into the type environment.
(2) **Body pass**: Recursively check all expression bodies, rule clauses, and proof blocks.

A `CheckContext` tracks whether the checker is currently inside a `proof`/`invariant` block (where temporal operators are permitted), a `rule` body, and the current temporal nesting depth (v1 limits to depth 1 to keep automaton construction tractable). The type checker enforces structural invariants including: temporal operators only in proof contexts; `when` clauses must be boolean; rate-limit windows must be `duration` typed; function argument counts match declarations.

The checker emits diagnostics into a `DiagnosticSink` rather than aborting on the first error, allowing all errors in a policy file to be reported in a single compilation pass (comparable to the Rust compiler's "multiple error" reporting model).

4.3 Lowering and State-Machine Construction

The lowering pass transforms the typed AST into `CompiledPolicy`, the IR that is serialized to `.aegisc` bytecode and consumed by the runtime.

Inheritance resolution. The lowering pass builds a policy inheritance map and resolves the `extends` chain for each policy (single-inheritance, so $O(d)$ where d is the inheritance depth). Inherited members are collected base-first.

Expression lowering. AST expressions are lowered to `IRExpr`, a type-erased, allocation-minimal representation suitable for tight evaluation loops. Variable references are resolved to slot indices (`RefRoot::Local(n)`). Match expressions are compiled to `DecisionTree` nodes for efficient dispatch.

Temporal to state machine. Each temporal invariant in a `proof` block is compiled to a `StateMachine` value. The construction depends on the outermost temporal operator:

- **`always(`ϕ`)`** Two-state machine: initial state s_0 (active/satisfied); transition to s_1 (violated, absorbing) on $\neg\phi$.
- **`never(`ϕ`)`** Two-state machine: initial state s_0 (active/satisfied); transition to s_1 (violated, absorbing) on ϕ.
- **`eventually(`ϕ`) within T`** Three-state machine: s_0 (waiting); transition to s_1 (satisfied, absorbing) on ϕ; transition to s_2 (violated, absorbing) on timeout T if still in s_0.
- **`until(`ϕ`, `ψ`)`** Three-state machine: s_0 (holding); transition to s_1 (released, absorbing) on ψ; transition to s_2 (violated, absorbing) on $\neg\phi \wedge \neg\psi$.
- **`after(`ϕ`, trigger)`** Four-state machine: s_0 (waiting for trigger); transition to s_1 (triggered) on *trigger*; from s_1, transition to s_2 (satisfied, absorbing) on ϕ, transition to s_3 (violated, absorbing) on $\neg\phi$. Because ϕ is evaluated on the event immediately following the trigger, this encodes $\Box(\mathit{trigger} \Rightarrow \mathcal{X}\phi)$.
- **`before(`ϕ`, `ψ`)`** Three-state machine: s_0 (waiting); transition to s_1 (satisfied, absorbing) on ϕ; transition to s_2 (violated, absorbing) on ψ. Enforces that ϕ occurs before ψ in the event stream ($\neg\psi\, \mathcal{U}\, \phi$).
- **`next(`ϕ`)`** Four-state machine: s_0 (initial); unconditional transition to s_1 (checking) on the next event; from s_1, transition to s_2 (satisfied, absorbing) on ϕ, transition to s_3 (violated, absorbing) on $\neg\phi$.

Figure 2 illustrates the `always` and `eventually within T` automata.

4.4 Bytecode Format

The `.aegisc` format is a self-describing binary envelope:

Field	Content	Size
Magic	`0xAE915C01`	4 bytes
Version	`0x0001`	2 bytes
Flags	reserved	2 bytes
Payload length	little-endian uint32	4 bytes
Payload	JSON-serialized `CompiledPolicy`	variable

The magic number encodes a mnemonic (`AEGIS`) and the major version in the low byte, enabling version detection without full deserialization. The JSON payload format is human-inspectable via the `aegisc inspect` subcommand, which aids policy debugging and audit.

5 Runtime Verifier

5.1 Event Evaluation

The `PolicyEngine` owns a deserialized `CompiledPolicy`, a vector of `StateMachineInstance` values, a map of `RateLimiter` values, and a `context` map of persistent per-session state. It exposes a single primary method:

```
evaluate(event: &Event) -> PolicyResult
```

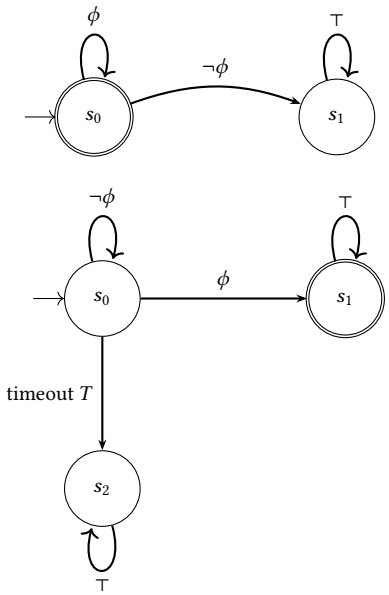

Figure 2: State machines for `always`(ϕ) (top) and `eventually`(ϕ) `within T` (bottom). s_0: active; s_1: satisfied (accepting); s_2: violated (absorbing). Double circles denote accepting/non-violating states.

The evaluation algorithm (`engine.rs`) runs in five steps:

(1) **Rate-limit check.** For each `CompiledConstraint`, invoke the corresponding `RateLimiter::record`. If any limiter exceeds its budget, return `Verdict::Deny` immediately.
(2) **Rule evaluation.** Iterate compiled rules in order. For each rule, check event-type membership (O(1) SmolStr comparison). If the rule applies, evaluate the optional `when` condition using the expression evaluator. Collect verdicts and actions from matching rules.
(3) **Verdict resolution.** `Deny` overrides all others; among the remainder, `Audit` and `Redact` override `Allow`.
(4) **State-machine stepping.** For each `StateMachineInstance` not in a terminal state, call `step(eval_ctx, now_ms)`. Collect any transitions to violating states.
(5) **Result construction.** Assemble `PolicyResult` with verdict, reason, triggered rule IDs, actions, violations, and evaluation latency in microseconds.

5.2 Expression Evaluator

The expression evaluator (`eval.rs`) recursively reduces `IRExpr` trees against an `EvalContext` holding the current event, session context, policy configuration, and local variable slots. Design goals are:

- **Allocation-minimal.** The hot path avoids heap allocation for scalar results. Collections allocate only when constructed by the expression, not on every traversal.
- **Short-circuit.** `&&` and `||` short-circuit as in standard evaluation. The `implies` operator (`a implies b`) is equivalent to `!a || b` and short-circuits when `a` is false.
- **Numeric coercion.** Cross-type comparisons between `Int` and `Float` values coerce `Int` to `Float` without panicking, matching the type system's subtyping rule.
- **Quantifier streaming.** `all`, `any`, `none`, and `exists` iterate their collection without materializing an intermediate result vector.

The runtime's `Value` type is an eight-variant enum covering `Null`, `Bool`, `Int(i64)`, `Float(f64)`, `String(SmolStr)`, `Duration(u64)`, `List(Vec<Value>)`, and `Map(HashMap<SmolStr, Value>)`.

5.3 State-Machine Execution

Each `StateMachineInstance` wraps the compiled `StateMachine` specification and tracks the current state ID and start timestamp. `step(ctx, now_ms)` evaluates transitions from the current state:

(1) If a `Timeout` guard is present and the elapsed time since `start_time_ms` exceeds `deadline_millis`, fire the timeout transition.
(2) Otherwise, evaluate each predicate-guarded transition in order; fire the first whose guard evaluates to true.

Violated and satisfied states are absorbing (self-loop with $\top$ guard), so the step function returns immediately once a terminal state is reached. This bounds per-event automaton work to $O(|\delta|)$ where $|\delta|$ is the number of transitions from the current state (typically 1–3).

5.4 Sliding-Window Rate Limiters

Each `CompiledConstraint` with `kind = RateLimit` or `kind = Quota` is paired with a `RateLimiter` that maintains a timestamp vector implementing a sliding window:

(1) Compute `window_start = now_ms - window_millis`.
(2) Evict all entries < `window_start` from the front of the timestamp vector (amortized $O(1)$ per call).
(3) Append `now_ms` to the vector.
(4) If `len() > limit`, return `exceeded`.

The sliding-window design ensures that a burst of exactly `limit` events at the boundary of two fixed windows (each window's count appearing at half the limit) cannot bypass enforcement. The memory cost is $O(L)$ per limiter where L is the limit value.

5.5 Audit Log

The audit log is an append-only ring buffer of `AuditEntry` values, each recording: policy name, event type, verdict, reason, triggered rules, violations, evaluation latency, and a monotonic timestamp. The ring buffer has a configurable maximum size; when full, the oldest entry is overwritten. The `total_recorded` counter is never decremented, enabling external auditors to detect gaps.

Entries can be queried by verdict type, event type, violation presence, or recency, and serialized to JSON for export to external SIEM systems. An optional file-flush path writes entries synchronously to a local append-only log file.

5.6 Fail-Closed Semantics

Any error in rule evaluation (type mismatch, missing field, division by zero) returns `Value::Null`, which is falsy; conditions that evaluate to `Null` are treated as unmatched. At the Python SDK layer, any exception from the Rust engine results in a deny verdict with an error reason. This fail-closed contract ensures that an unanticipated policy input cannot silently bypass enforcement.

6 Python SDK

AutomaGuard provides a Python SDK built on pyo3, enabling native-performance policy enforcement within Python-based agent frameworks. The SDK design principle is "Python as skin, not brain": all evaluation logic lives in Rust; the Python layer is a thin adapter.

6.1 Core API

```python
from aegis_enforce import enforce

# Wrap any OpenAI or LangChain client
safe_client = enforce(client, policy="guard.aegisc")

# Use identically to the unwrapped client
response = safe_client.chat.completions.create(...)
```

Listing 3: Drop-in enforcement with three lines.

The `enforce()` function returns a proxy object that intercepts tool calls, evaluates them against the loaded policy, and either passes them through or raises `EnforcementError` depending on the verdict. An `audit` verdict passes through but records the event; a `redact` verdict sanitizes the response before returning it to the caller.

6.2 Framework Integrations

OpenAI.. The OpenAI integration wraps `openai.OpenAI` with a proxy that intercepts `chat.completions.create` calls. Tool calls embedded in assistant messages are extracted, evaluated as `tool_call` events, and blocked if denied.

LangChain. The LangChain integration provides an `AutomaGuardCallbackHandler` implementing the `BaseCallbackHandler` interface. Hooks fire on `on_tool_start`, `on_tool_end`, and `on_tool_error`, converting LangChain's callback arguments to AutomaGuard events and raising `EnforcementError` on denial.

6.3 Direct Engine Access

For frameworks not yet natively integrated, the `PolicyEngine` class is exposed directly:

```python
from aegis_enforce import PolicyEngine

engine = PolicyEngine.from_file("guard.aegisc")
result = engine.evaluate("tool_call", {
    "tool_name": "send_email",
    "arguments": {"to": "user@example.com",
                  "body": context["draft"]}
})
if result["verdict"] == "deny":
    raise RuntimeError(result["reason"])
```

Listing 4: Direct engine access.

The `evaluate()` method returns a plain Python dictionary, keeping the API dependency-free for callers that do not use the framework-specific wrappers.

7 Evaluation

7.1 Latency Benchmarks

We evaluate runtime latency using Criterion.rs (a statistically rigorous benchmarking library) on a Linux workstation. Table 1 reports mean latency and headroom under the 10 ms budget across seven policy scenarios.

Table 1: Per-event evaluation latency (Criterion.rs mean). R = rules, S = state machines, L = rate limiters.

Scenario	R	S	L	Mean latency
Baseline (empty policy)	0	0	0	38 ns
Single allow rule	1	0	0	104 ns
Field condition rule	1	0	0	104 ns
5 rules	5	0	0	750 ns
4 state machines	0	4	0	1.4 µs
Rate limiter only	0	0	1	104 ns
Realistic (10R+2S+1L)	**10**	**2**	**1**	**2.4 µs**

The realistic scenario (10 rules, 2 state machines, 1 rate limiter) achieves 2.4 µs mean latency on the full evaluation path (compound conditions, all 10 rules scanned)—a factor of 4,166× under the 10 ms budget. For allow-listed events that exit after the first matching rule, steady-state throughput on the same policy exceeds 400,000 events per second on a single thread.

A sustained 100-event stream completes in approximately 535 µs total (5.4 µs per event), reflecting the amortized cost of the sliding-window eviction.

Why the latency budget is achievable. The key design decision enabling these numbers is compilation: because `.aegisc` policies are deserialized once at engine initialization, the hot path in `evaluate()` involves no string parsing, no regular expression compilation, and no memory allocation for the common case. Rule evaluation is a linear scan with short-circuit; state-machine stepping is a small transition table lookup; rate-limit checking is a timestamp-vector prefix eviction. All of these are cache-friendly operations on data structures sized to fit in L1 for realistic policies.

7.2 Policy Expressiveness

We demonstrate that the Aegis language can encode the full range of agent safety properties identified in prior work [3, 8]:

Forbidden actions. `never(event.tool_name == "drop_table")`

Required approvals. `eventually(human_reviewed) within 24h`

Ordered sequences. `after(send_email, pii_access)` (deny if email follows PII read without intervening sanitization)

Budget enforcement. `rate_limit api_calls: 100 per 1m`

Data classification gates. `when event.classification == "PII" deny` with quantifiers over structured arguments

Path traversal prevention. String predicate `event.path contains ".."`

The `data_exfiltration_guard.aegis` example policy (142 lines) encodes 15 distinct safety properties spanning all six categories above. The `research_guard.aegis` example (98 lines) specifically demonstrates the value of temporal composition: five of its invariants detect attack *sequences* that would be undetectable by per-event rules alone.

7.3 Compiler Test Coverage

The aegis-compiler crate contains 492 tests (152 unit, 340 integration) achieving 85.2% line coverage. Coverage by module:

Module	Line coverage
`ast/nodes`	100%
`ast/span`	100%
`ir`	99.2%
`bytecode`	95.7%
`diagnostics`	97.1%
`parser`	86.4%
`lower`	82.8%
`checker`	61.5%

The runtime crate contains 184 tests covering expression evaluation (118 tests), engine behavior (36 tests), and audit log properties (30 tests). The Python SDK contains 91 tests exercising the pyo3 bridge, enforcement logic, and both framework integrations.

8 Related Work

Runtime guardrails. NeMo Guardrails [9] provides a declarative Colang language for specifying conversation flows and safety rules, evaluated by a secondary LLM call at runtime. LlamaGuard [5] fine-tunes a model for content classification. Guardrails AI defines Python validators applied to LLM outputs. These systems interpret or run inference at request time; AutomaGuard compiles ahead of time. None supports temporal invariants over event sequences.

Static workflow verification. Xavier et al. [13] present a system that extracts abstract graph models from LangGraph, CrewAI, AutoGen, and Google ADK and applies structural checks and a temporal policy DSL (compiled to DFAs) for pre-deployment verification. This is complementary to AutomaGuard: static analysis guarantees path coverage but cannot condition on runtime data. Their DSL targets the safety fragment of LTL (DFAs); Aegis additionally supports liveness properties via Büchi-like constructions for `eventually` and `until`.

LTL runtime monitoring. There is a large literature on runtime verification of LTL properties [4, 7]. MarQ [10] monitors Java execution traces using Quantified Event Automata (QEA) as the specification formalism. AutomaGuard adapts the core automaton construction to the specific domain of AI agent events: a typed, heterogeneous event stream with arbitrary structured fields, requiring the composition of temporal and data predicates.

Policy languages for systems. Rego (Open Policy Agent [11]) is a Datalog-inspired policy language for access control. Cedar [1] is a purpose-built attribute-based access control language with SMT-backed static analysis. Both operate on individual decision requests and do not model event sequences. AutomaGuard's Aegis provides temporal operators and an agent-specific event model not present in either.

Formal verification of AI systems. DNN verification tools (Marabou [6], α, β-CROWN [12, 14]) verify input-output properties of the function computed by a deployed neural network, not behavioral policies over agent event streams. AI safety frameworks such as Constitutional AI [2] operate at the training level. AutomaGuard is complementary: it enforces behavioral constraints on a deployed system without access to model internals.

9 Discussion and Limitations

What AutomaGuard does not verify. Temporal invariants over the event stream provide sequence-level guarantees conditioned on observable event fields. AutomaGuard does not verify:

- Semantic correctness of LLM-generated content (factuality, intent).
- Properties of model weights or internal activations.
- Correctness of the agent framework's event generation (if the framework fails to emit an event for a tool call, AutomaGuard cannot enforce against it).

Trust boundary. The guarantees are as strong as the event hook. Integrations that instrument at the framework callback level (LangChain's `BaseCallbackHandler`) rely on the framework emitting complete, accurate events. A future MCP (Model Context Protocol) proxy interceptor would enforce at the transport level, removing this assumption.

Automata completeness. The current state-machine construction for `eventually` uses a deadline-based timeout rather than a true Büchi accepting condition. This is sound for safety-critical enforcement (a missed deadline is a violation), but does not handle liveness properties over infinite traces in the classical sense. Full Büchi acceptance is a target for v2.

Policy composition. Aegis v1 supports single-inheritance policy composition. Multi-policy enforcement (running several compiled policies simultaneously on the same event stream) is achievable by running multiple `PolicyEngine` instances in parallel, but is not yet abstracted into the SDK.

10 Conclusion

We have presented AutomaGuard, a formal policy enforcement engine for production AI agents. The central thesis is that *compiled* enforcement—translating safety policies to state machines ahead of

deployment—makes it possible to enforce temporal invariants over agent event streams within a hard latency budget, something that interpreted guardrails and pre-deployment static analyzers cannot both simultaneously provide.

The Aegis Policy Language provides a typed DSL with first-class temporal operators (`always`, `eventually`, `never`, `until`, `after`) that compose with arbitrary data predicates over structured event fields. The Rust runtime verifier evaluates compiled policies at 2.4 µs mean latency on a realistic policy—more than 4,000× under the 10 ms per-event budget. A Python SDK with LangChain and OpenAI integrations enables drop-in adoption with three lines of user code.

AutomaGuard is available as open-source software.

Acknowledgments

[Acknowledgements redacted for review.]

References

[1] Amazon Web Services. 2023. Cedar Policy Language. https://www.cedarpolicy.com.

[2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022. Constitutional AI: Harmlessness from AI Feedback. *arXiv preprint arXiv:2212.08073* (2022).

[3] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. ACM.

[4] Klaus Havelund and Grigore Roşu. 2004. Efficient Monitoring of Safety Properties. *International Journal on Software Tools for Technology Transfer* 6, 2 (2004), 158–173.

[5] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674* (2023).

[6] Guy Katz, Derek A. Huang, Duligur Ibeling, Kyle Julian, Christopher Lazarus, Rachel Lim, Parth Shah, Shantanu Thakoor, Haoze Wu, Aleksandar Zeljic, David L. Dill, Mykel J. Kochenderfer, and Clark W. Barrett. 2019. The Marabou Framework for Verification and Analysis of Deep Neural Networks. In *Proceedings of the 31st International Conference on Computer Aided Verification (CAV 2019) (Lecture Notes in Computer Science, Vol. 11561)*. Springer, 443–452.

[7] Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303.

[8] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. *arXiv preprint arXiv:2211.09527* (2022).

[9] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 431–445.

[10] Giles Reger, Helena Chockler, and David Rydeheard. 2015. MarQ: Monitoring at Runtime with QEA. In *Proceedings of the 21st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2015) (Lecture Notes in Computer Science, Vol. 9035)*. Springer, 596–610.

[11] Torin Sandall, Tim Hinrichs, et al. 2021. Open Policy Agent. https://www.openpolicyagent.org.

[12] Shiqi Wang, Huan Zhang, Kaidi Xu, Xue Lin, Suman Jana, Cho-Jui Hsieh, and J. Zico Kolter. 2021. Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Complete and Incomplete Neural Network Robustness Verification. In *Advances in Neural Information Processing Systems (NeurIPS 2021)*, Vol. 34.

[13] Melwin Xavier, Vaisakh M A, Melveena Jolly, and Midhun Xavier. 2026. Agentproof: Static Verification of Agent Workflow Graphs. *arXiv preprint arXiv:2603.20356* (2026).

[14] Kaidi Xu, Zhouxing Shi, Huan Zhang, Yihan Wang, Kai-Wei Chang, Minlie Huang, Bhavya Kailkhura, Xue Lin, and Cho-Jui Hsieh. 2021. Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers. In *International Conference on Learning Representations (ICLR 2021)*.